

Lectura 6.1: Operadores Neuronales, DeepONets y PI-DeepONets

Introducción a las redes neuronales informadas por física
Especialización en Inteligencia Artificial - B32025

Outline

- Funciones, funcionales y operadores
- Teoremas universales de aproximación
- DeepONets
- PI-DeepONets

Funciones, funcionales y operadores

	Entrada	Salida	Ejemplo
Función	Números, vectores, etc	Números, vectores, etc	$f(x) = x^2$ $g(x, y) = x + y$
Funcional	Funciones	Números	$F[f] = \int_a^b f(x) dx.$ $F[f] = \ f\ = \sqrt{\int_a^b f(x) ^2 dx}.$
Operador	Funciones	Funciones	$T[f] = \frac{df}{dx}.$ $T[f](x) = \int_a^x f(t) dt.$

Aproximación universal para funcionales

Theorem

Suppose that U is a compact set in $C[a, b]$, f is a continuous functional defined on U , and $\sigma(x)$ is a bounded generalized sigmoidal function, then for any $\epsilon > 0$, there exist $m + 1$ points $a = x_0 < \cdots < x_m = b$, a positive integer N and constants $W_i^1, \theta_i, W_{i,j}^2, i = 1, \cdots, N, j = 0, 1, \cdots, m$, such that

$$\left| f(u) - \sum_{i=1}^N W_i^1 \sigma \left(\sum_{j=0}^m W_{i,j}^2 u(x_j) + \theta_i \right) \right| < \epsilon$$

Aproximación universal para operadores

$$G : u \mapsto G(u), G(u) : y \in \mathbb{R}^d \rightarrow \mathbb{R}$$

Theorem (Chen & Chen, IEEE Trans. Neural Netw., 1995)

Suppose that σ is a continuous non-polynomial function, X is a Banach Space, $K_1 \subset X, K_2 \subset \mathbb{R}^d$ are two compact sets in X and $\mathbb{R}^d$, respectively, V is a compact set in $C(K_1)$, G is a nonlinear continuous operator, which maps V into $C(K_2)$. Then for any $\epsilon > 0$, there are positive integers n, p, m , constants $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in \mathbb{R}, w_k \in \mathbb{R}^d, x_j \in K_1, i = 1, \dots, n, k = 1, \dots, p, j = 1, \dots, m$, such that

$$|G(u)(y) - \underbrace{\sum_{k=1}^p \sum_{i=1}^n c_i^k \sigma \left(\sum_{j=1}^m \xi_{ij}^k u(x_j) + \theta_i^k \right)}_{\text{branch}} \underbrace{\sigma(w_k \cdot y + \zeta_k)}_{\text{trunk}}| < \epsilon$$

holds for all $u \in V$ and $y \in K_2$.

T.P. Chen and H. Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. IEEE Transactions on Neural Networks, 6(4):911–917, 1995.

Aproximación universal para operadores mediante DeepONets

Theorem (Lu et al, Nature Machine Intelligence 2021)

Let X be a Banach Space and $K_1 \subset X$ be compact. Let $V \subset C(K_1)$ be compact and $K_2 \subset \mathbb{R}^d$ be compact. Assume that $\mathcal{G} : V \rightarrow C(K_2)$ is a nonlinear continuous operator. Then for any $\epsilon > 0$, there are positive integers n, p, m , constants $c_i^k \in \mathbb{R}$, neural networks $f^{\mathcal{N}}(\cdot; \theta^{(k)})$ and $g^{\mathcal{N}}(\cdot; \Theta^{(k,i)}), x_j \in K_1, i = 1, \dots, n, k = 1, \dots, p, j = 1, \dots, m$, such that

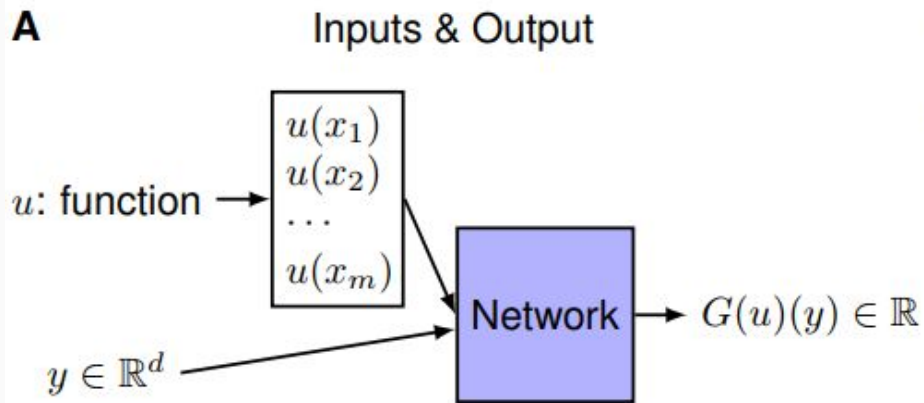
$$|\mathcal{G}(u)(y) - \underbrace{\sum_{k=1}^p \sum_{i=1}^n c_i^k g^{\mathcal{N}}(u_m; \Theta^{(k,i)})}_{\text{branches}} \underbrace{f^{\mathcal{N}}(y; \theta^{(k)})}_{\text{trunk}}| < \epsilon$$

holds for all $u \in V$ and $y \in K_2 \subseteq \mathbb{R}^d$, where

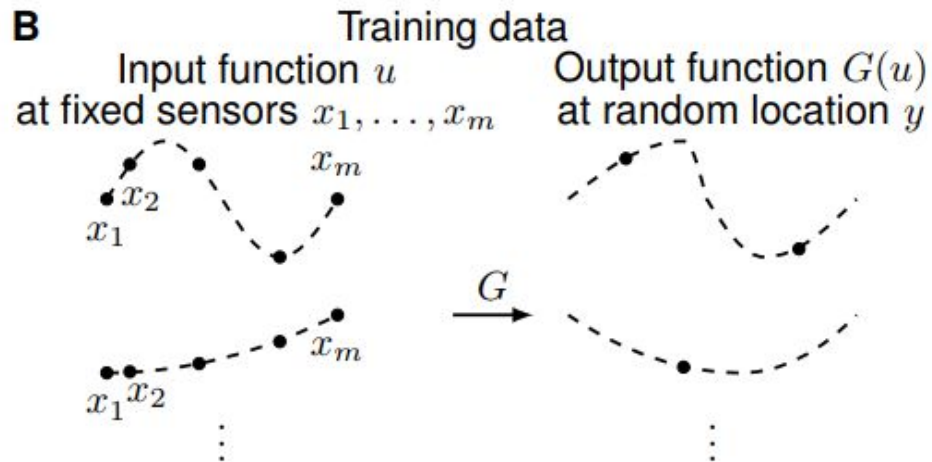
- $u_m = [u(x_1), u(x_2), \dots, u(x_m)]^\top \in \mathbb{R}^m$,
- and the neural networks $f^{\mathcal{N}}(y; \theta^{(k)})$ and $g^{\mathcal{N}}(u_m; \Theta^{(k,i)})$ satisfy the universal approximation theorem of continuous functions on compact sets.

Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence. 2021 Mar;3(3):218-29

DeepONets - Formulación

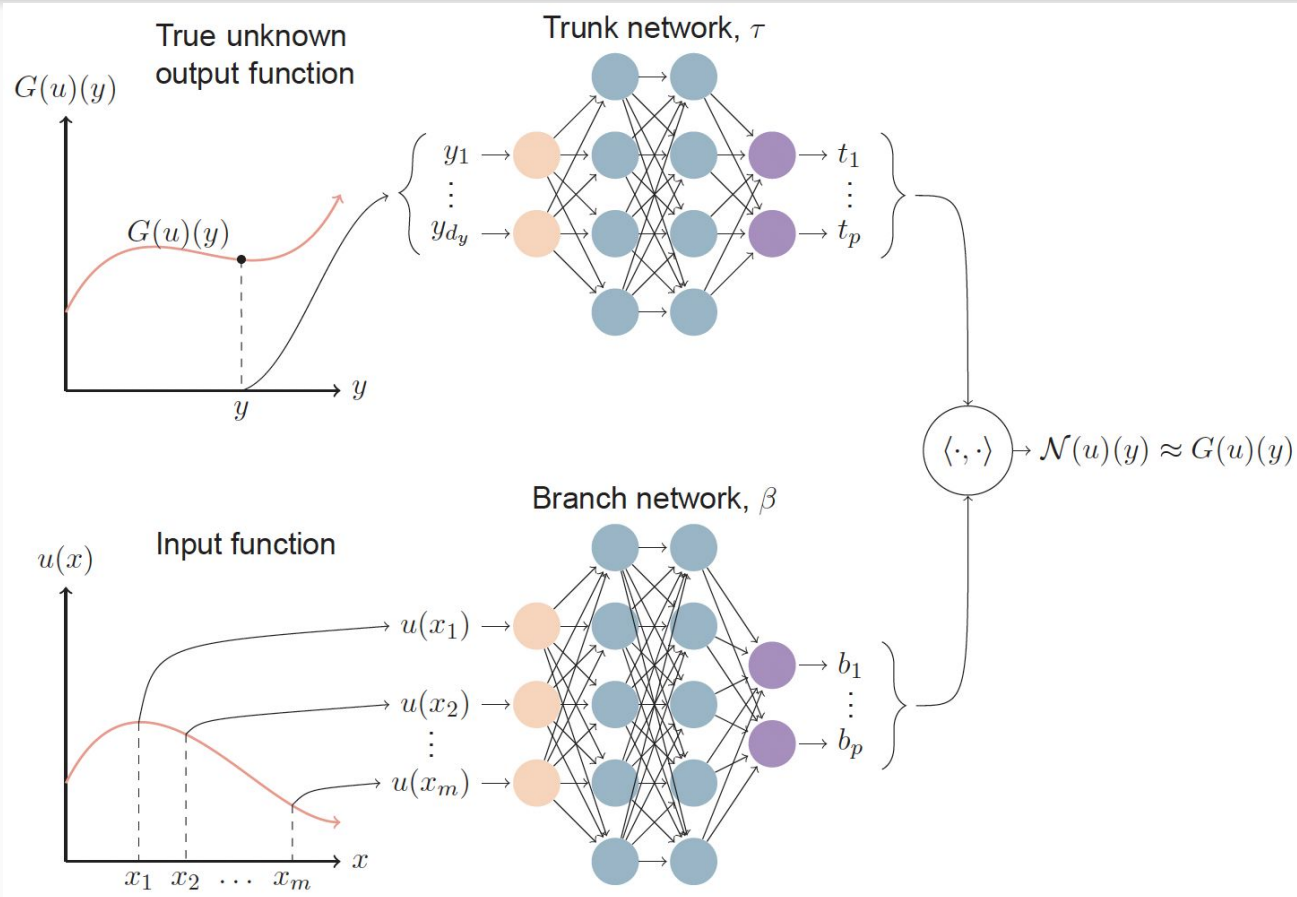


$u \in V$ and $y \in K_2$.



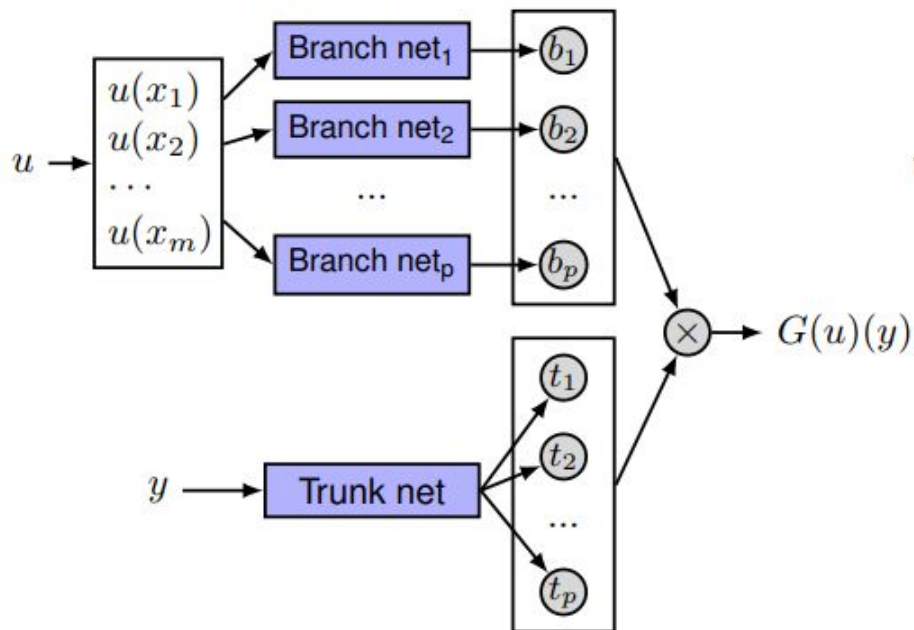
Operador neuronal: $G : u \mapsto G(u)$

DeepONets - Formulación

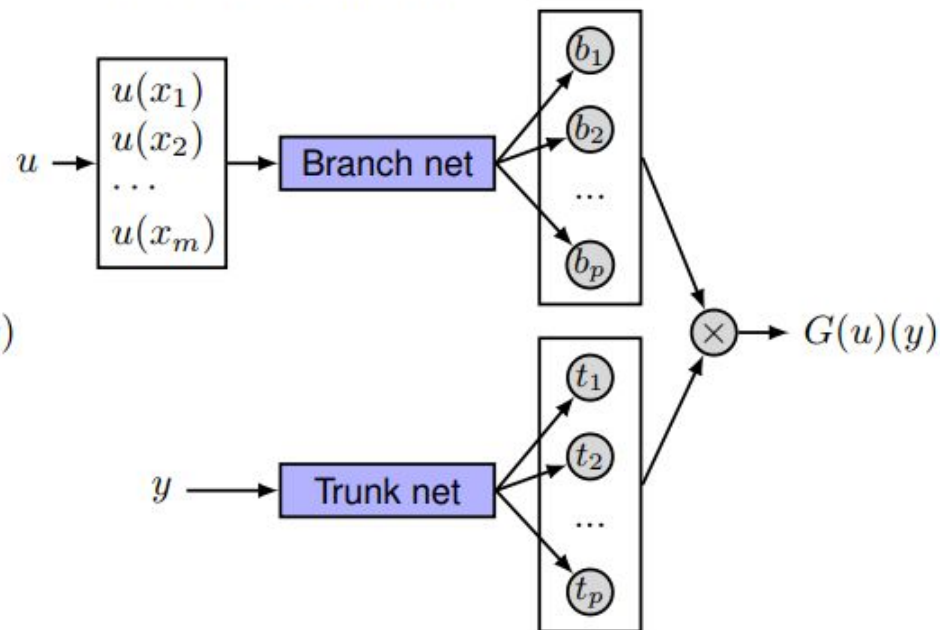


DeepONets - Formulación

C Stacked DeepONet



D Unstacked DeepONet



DeepONets - Formulación

- Permiten aproximar operadores lineales y no lineales
- Constan de dos redes neuronales:
 - **Branch:** codifica el espacio de funciones de input discreto (toma la función discretizada $v \in D$ como input)
 - **Trunk:** codifica el dominio de las funciones de output (toma las coordenadas $\xi \in D'$ como input)
- Las arquitecturas branch y trunk pueden ser de cualquier tipo
- Salida del modelo:

$$G(u)(y) \approx \sum_{k=1}^p b_k t_k + b_0.$$

donde b_0 es el bias $\in \mathbb{R}$, $\{b_1, b_2, \dots, b_p\}$ son las p outputs de la red branch, mientras que $\{t_1, t_2, \dots, t_p\}$ son las p outputs de la red trunk

Ejemplo: DeepONet para el operador anti-derivada

- Consideremos la ODE dada por:

$$\frac{du(x)}{dx} = g(u(x), v(x), x), \quad x \in [0, 1],$$

con condición de borde $u(0)=0$. El objetivo es predecir $u(x)$ sobre el dominio $[0, 1]$ para cualquier función $v(x)$.

- Consideremos un problema lineal al hacer $g(u(x), v(x), x) = v(x)$, es decir:

$$\begin{aligned} \frac{du(x)}{dx} &= v(x), \quad x \in [0, 1], \quad G : v(x) \rightarrow u(x), \\ u(x) &= u_0 + \int_0^x v(\tau) d\tau, \quad x \in [0, 1] \end{aligned}$$

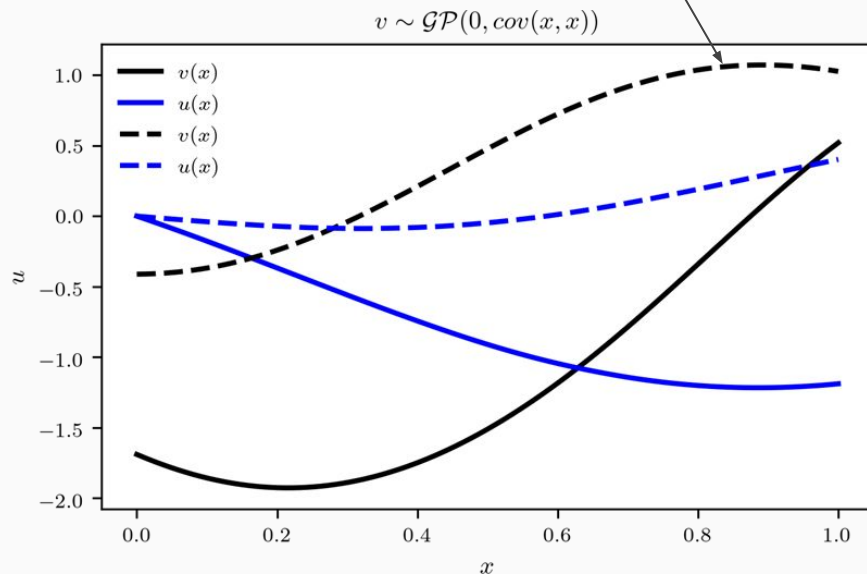
Ejemplo: DeepONet para el operador anti-derivada

- Samplear $v(x) \sim GP(0, cov(x, x))$
- Para la $v(x)$ sampleada, computar el siguiente problema de valor inicial mediante cualquier esquema numérico:

$$\frac{du(x)}{dx} = v(x) \quad x \in [0, 1]$$

- La imagen muestra ejemplos de dos $u(x)$ computadas a partir de dos $v(x)$ sampleadas

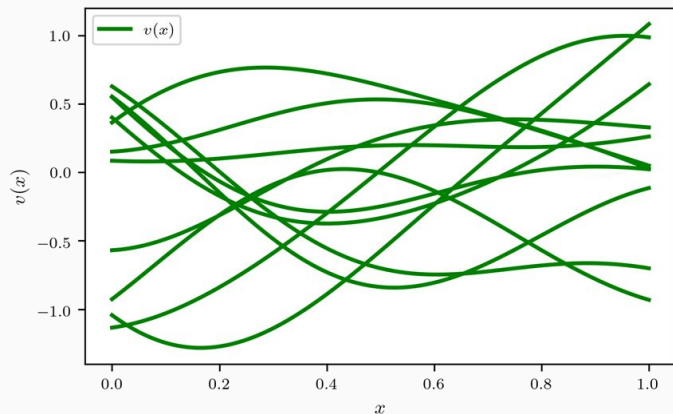
Función de entrada $v(x)$ sampleada en una grilla de puntos fija $x_1, x_2, \dots, x_p$



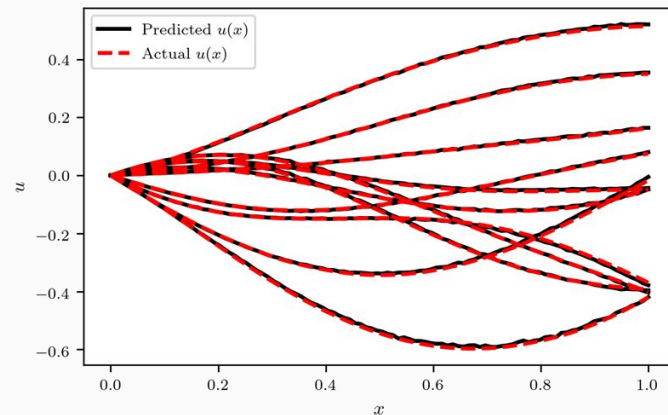
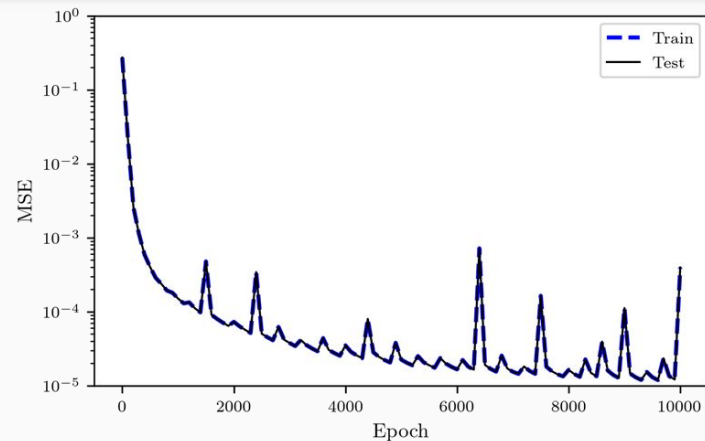
Ejemplo: DeepONet para el operador anti-derivada

Resultados:

- Error de generalización: prácticamente nulo!
- El operador aprendido permite obtener la función de salida “ u ” *evaluada en un conjunto de puntos “ y ” para cualquier función de entrada “ v ” evaluada en una grilla fija de puntos “ x ”*



$$G(v)(y) = u(y)$$



DeepONets - Implementación

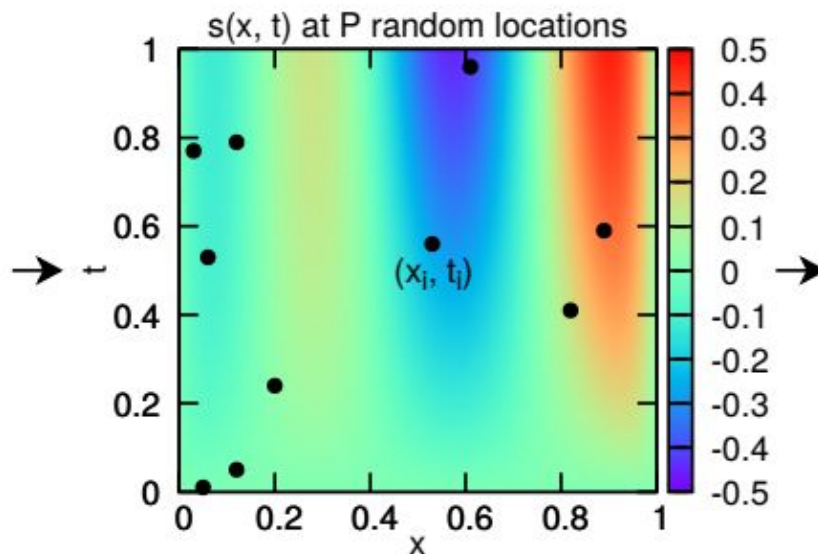
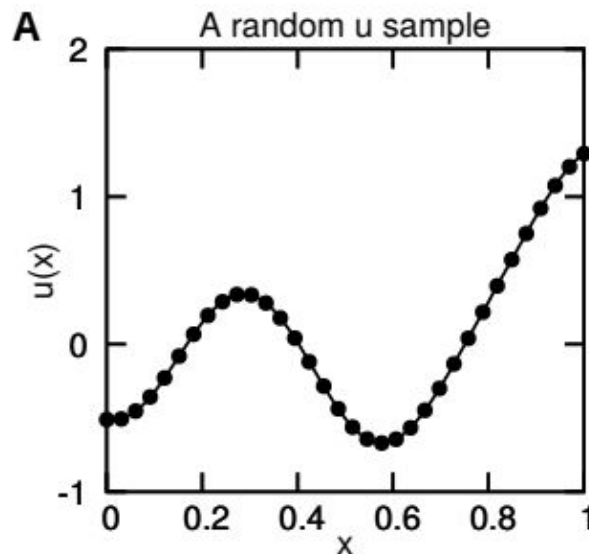
Data: Train Data: antiderivative_train.npz
Test Data: antiderivative_test.npz

Vamos al [colab](#)...

DeepONets - Difusión-reacción con término fuente

$$\frac{\partial s}{\partial t} = D \frac{\partial^2 s}{\partial x^2} + k s^2 + u(x), \quad x \in [0, 1], t \in [0, 1],$$

con CB y CI homogéneas.



- Training data from one u

$(u, (x_1, t_1), s(x_1, t_1))$

$(u, (x_2, t_2), s(x_2, t_2))$

...

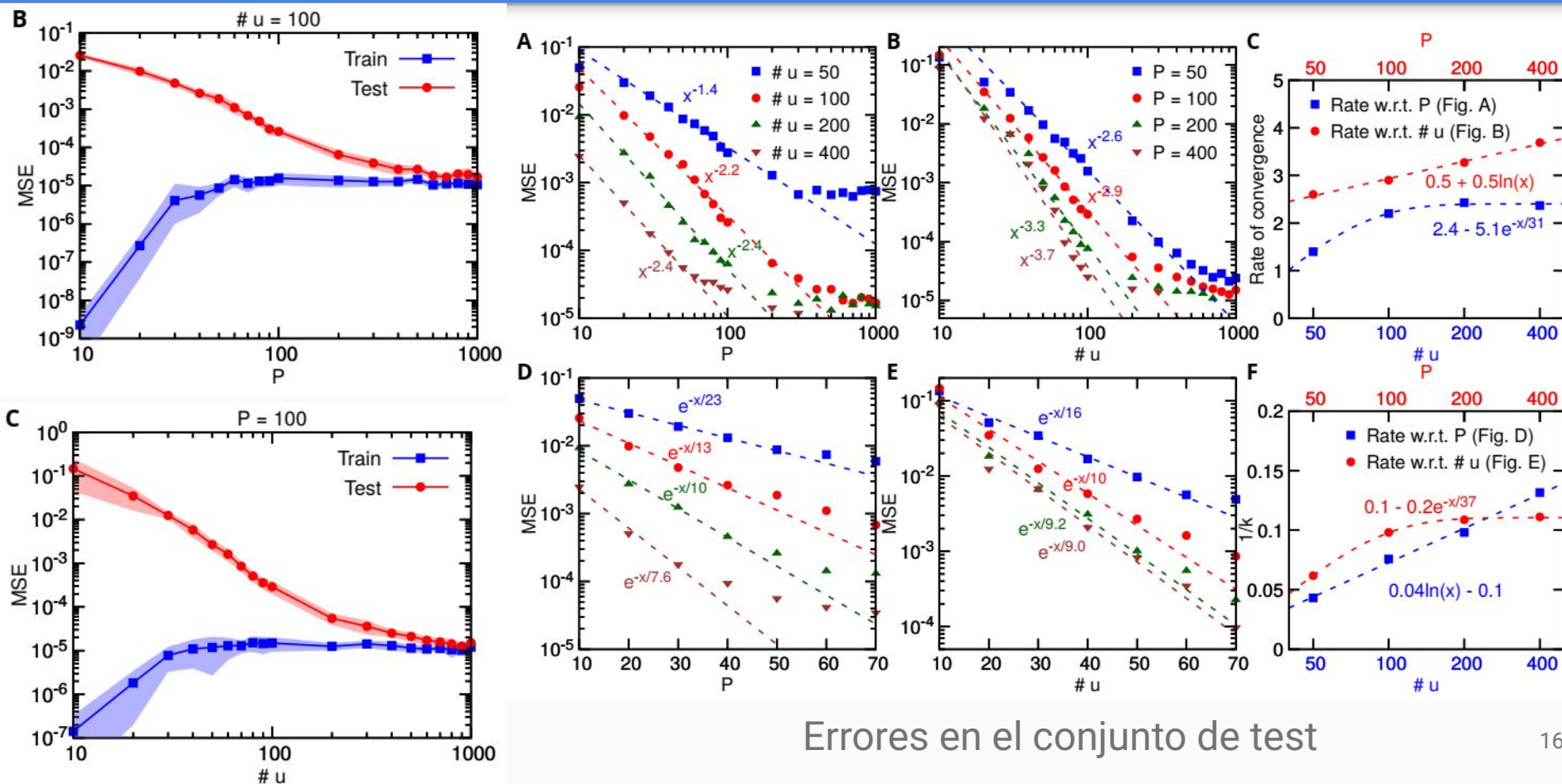
$(u, (x_P, t_P), s(x_P, t_P))$

- Repeat for different u

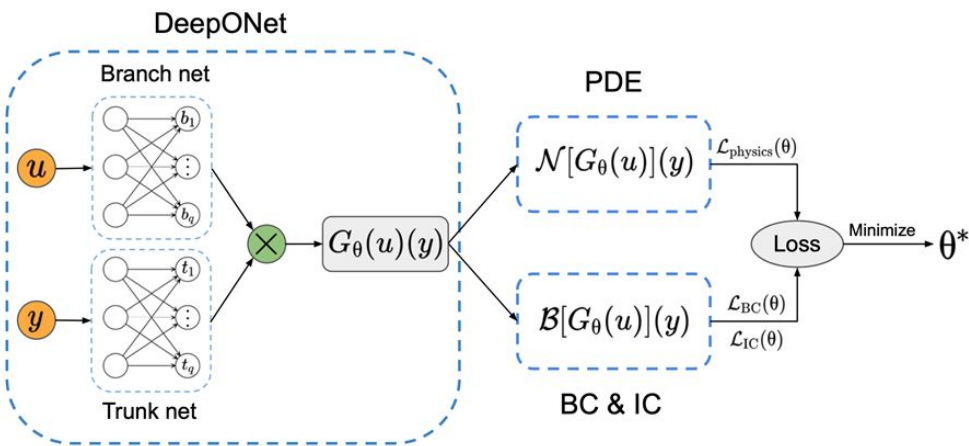
Se definen “# u ” funciones de entrada $u(x)$

Se definen “ P ” puntos de colocación

DeepONets - Difusión-reacción con término fuente



PI-DeepONets - Formulación



Salida de la DeepONet:

$$G_{\theta}(\mathbf{u})(\mathbf{y}) = \sum_{k=1}^q \underbrace{b_k(\mathbf{u}(\mathbf{x}_1), \mathbf{u}(\mathbf{x}_2), \dots, \mathbf{u}(\mathbf{x}_m))}_{\text{branch}} \underbrace{t_k(\mathbf{y})}_{\text{trunk}}$$

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{operator}}(\theta) + \mathcal{L}_{\text{physics}}(\theta)$$

donde:

$$\mathcal{L}_{\text{operator}}(\theta) = \frac{1}{NP} \sum_{i=1}^N \sum_{j=1}^P \left| G_{\theta}(\mathbf{u}^{(i)})(\mathbf{y}_{u,j}^{(i)}) - G(\mathbf{u}^{(i)})(\mathbf{y}_{u,j}^{(i)}) \right|^2$$

$$\mathcal{L}_{\text{physics}}(\theta) = \frac{1}{NQm} \sum_{i=1}^N \sum_{j=1}^Q \sum_{k=1}^m \left| \mathcal{N}(u^{(i)}(\mathbf{x}_k), G_{\theta}(\mathbf{u}^{(i)})(\mathbf{y}_{r,j}^{(i)})) \right|^2$$

Notar que la pérdida de la PDE se computa en base a las derivadas de la salida de la DeepONet con respecto a las entradas de la red trunk.

Wang S, Wang H, Perdikaris P. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. Science advances. 2021 Sep 29;7(40):eabi8605.

PI-DeepONets - Resultados interesantes

Table 1. Summary of benchmarks for assessing the performance of physics-informed DeepONets across various types of parametric differential equations. The reported test error corresponds to the relative L^2 prediction error of the trained model, averaged over all examples in the test dataset (see eq. S20).

Governing law	Equation form	Random input	Test error
Linear ODE	$\frac{ds(x)}{dx} = u(x)$	Forcing terms	$0.33 \pm 0.32\%$
Diffusion reaction	$\frac{\partial s}{\partial t} = D \frac{\partial^2 s}{\partial x^2} + k s^2 + u(x)$	Source terms	$0.45 \pm 0.16\%$
Burgers'	$\frac{\partial s}{\partial t} + s \frac{\partial s}{\partial x} - \nu \frac{\partial^2 s}{\partial x^2} = 0$	Initial conditions	$1.38 \pm 1.64\%$
Advection	$\frac{\partial s}{\partial t} + u \frac{\partial s}{\partial x} = 0$	Variable coefficients	$2.24 \pm 0.68\%$
Eikonal	$\ \nabla s\ _2 = 0$	Domain geometries	$0.42 \pm 0.11\%$

Wang S, Wang H, Perdikaris P. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. Science advances. 2021 Sep 29;7(40):eabi8605.

PI-DeepONets - Implementación

Resolvemos el problema de conducción de calor para obtener la temperatura “u” en un tiempo “t>0” a partir de la condición inicial $u_0 = u(x, t=0)$:

$$G(u_0)(x, t) = u(x, t)$$

Vamos al [colab](#)...

DeepONets - Conclusiones/Comentarios finales

- Las NN son aproximadores universales de funciones, funcionales y operadores no lineales
- Las DeepONet se diseñaron en base al teorema de Chen & Chen y fueron extendidas a DNN
- Las DeepONet convergen exponencialmente con los datos de entrenamiento, pero en la práctica saturan con Big Data
- Las DeepONet pueden aprender operadores multiescala
- Se han desarrollado argumentos teóricos rigurosos que respaldan las DeepONets (por ejemplo, demuestran ser robustas frente a ruido)
- Las PI-DeepONet proveen mejor generalización y precisión
- No son robustos frente a sistemas dinámicos que tiendan a ser caóticos

¿Dudas o
preguntas?